

Builder Journal

What worked well?

- **Modular architecture:** splitting the system into separate modules (document processing, embeddings, vector store, LLM, chat memory) meant I could test each piece on its own with plain sample text and a fake embedding function before ever running the full app. Bugs became much easier to isolate this way.
- **Offline-first fallback design:** building a local TF-IDF + extractive-answer mode alongside the Gemini-powered mode meant the app was demoable and testable from the very first run, without waiting on an API key, quota, or internet access.
- **ChromaDB for storage:** native metadata filtering made the "search specific documents only" requirement almost trivial to implement, and persistence meant uploaded documents survived app restarts without extra work.
- **Reusing earlier experiments:** the chunk size/overlap defaults (500 characters, 10% overlap) came directly from the measurements in Assignment 4 rather than guesswork, which made those design decisions easy to justify.

What challenges did you face?

- **Embedding dimension mismatch:** switching between the local (512-dim) and Gemini (3072-dim) embedding providers crashed ChromaDB with a dimension error, because both were writing into the same collection. The fix was to give each provider its own collection so vectors never mix dimensions.
- **A deprecated model, not a broken key:** the Gemini calls failed with a quota error showing limit: 0. It looked like a billing problem, but a standalone test script proved the API key itself worked fine for embeddings only text generation on gemini-2.0-flash failed. Research confirmed Google had deprecated that exact model in mid-2026. Swapping to the current gemini-2.5-flash model resolved it immediately.
- **Inconsistent "no page number" handling:** TXT/DOCX/Markdown files have no real page concept, so I stored a placeholder value internally. One display panel showed that raw placeholder ("page -1") instead of the friendly "N/A" used everywhere else a small but visible inconsistency caught during manual testing.
- **An unhandled API failure could crash the whole app:** the first version let a Gemini quota error propagate all the way up through Streamlit as a raw traceback, which is a poor experience mid-demo. Every LLM call now needs to fail safely without taking the rest of the UI down with it.

How did you debug problems?

- **Isolated standalone scripts:** rather than debugging inside the full Streamlit app, I wrote a small test_gemini_key.py script that called only the embedding and generation endpoints directly. This separated "is my key/account broken" from "is my app code broken" in seconds.
- **Read the full traceback, not just the last line:** the dimension-mismatch and quota errors both looked cryptic at first glance, but the underlying chromadb and google-genai library error messages,

once read fully, pointed directly at the real cause (collection dimension, then quota metric and model name).

- **Verified assumptions against current documentation:** rather than assuming the Gemini model name from memory, I checked Google's current pricing/deprecation page once the quota error looked structural (limit: 0) rather than transient which confirmed the model shutdown date and the replacement model name.
- **Reproduced bugs with minimal synthetic data:** small, hand-written test documents (a two-sentence remote-work policy, a two-sentence leave policy) made it fast to confirm retrieval, citations, and deletion behaved correctly without waiting on large file uploads.

What would you improve?

- **Config-driven model names:** the Gemini model name is currently hardcoded; moving it to a config value would make future model deprecations a one-line change instead of a code edit.
- **Automated tests:** testing was done manually and interactively during development. A small pytest suite covering chunking, retrieval, and citation formatting would catch regressions automatically instead of relying on me noticing them.
- **Retry/backoff for transient API errors:** quota and rate-limit errors currently fail immediately; a short automatic retry with backoff would smooth over brief, transient 429s without bothering the user.
- **Clearer first-run guidance:** several early issues (missing .env, stale collections from an old embedding provider) would have been avoided with a startup check that validates configuration before the UI loads.

What did you learn about RAG?

Building this project made clear that RAG quality depends as much on the surrounding engineering as on the model itself. Chunk size and overlap directly control the precision/context trade-off at retrieval time exactly what Assignment 4's experiments measured, and this project made that trade-off tangible rather than theoretical. Citations and "insufficient evidence" behavior matter operationally, not just conceptually: without them, a RAG app can sound confident while being wrong, which is precisely the failure mode hallucination-reduction techniques exist to prevent.

I also learned that a production RAG system has real external dependencies that behave like infrastructure, not just an algorithm choice: embedding models fix a vector's dimension permanently for a collection, and LLM providers deprecate models and change quotas on their own schedule, independent of the application code. A robust RAG system has to plan for both through per-provider storage separation and graceful error handling rather than assuming the embedding model or LLM endpoint will remain constant over the project's lifetime.